Analyse de logs Linux et investigation d'incidents

Démarche complète : manuelle, automatisée et SIEM (Splunk)

Analyste	Jean-Hadley CHERY
Date du rapport	20 octobre 2026
Source	`Linux_2k.log` — 2 000 lignes de journaux système du serveur `combo`
Période couverte	du 14 juin 15:16:01 au 26 juillet 07:04:12, soit 42,7 jours
Outils	lecture directe (VS Code) · Excel et Power Query · script Python 3 · Splunk Enterprise 10.6
Environnement	Splunk Enterprise 10.6.0.5 installé et administré par mes soins sur un poste Windows 11, service en démarrage automatique, console locale sur le port 8000
Portée de l'installation	au-delà de cet exercice, la même instance reçoit les journaux de quatre machines du laboratoire par agents *Universal Forwarder* — 153 880 évènements collectés
Conduite du travail	quatre séances, du 17 au 20 octobre 2026

Cadre de l'exercice

Les journaux analysés proviennent de **LogHub**, une bibliothèque publique de jeux de données utilisée dans la recherche en cybersécurité. Ce sont des journaux réels, anonymisés, issus d'un serveur Linux exposé sur Internet.

Je le dis d'entrée : aucun système n'a été attaqué pour produire ce rapport, et aucune donnée personnelle n'est traitée. Ce qui est transposable, en revanche, c'est la méthode — identique à celle d'une investigation sur un incident réel, du premier coup d'œil au journal jusqu'aux recommandations remises à l'exploitant.

Résumé exécutif

Le serveur `combo` a subi, pendant les six semaines que couvrent ces journaux, une campagne de force brute SSH automatisée et distribuée.

J'ai relevé **490 échecs d'authentification** provenant de **47 adresses sources distinctes**. Le rythme atteint dix tentatives dans une même seconde et cinquante tentatives en une minute : aucun opérateur humain ne saisit des mots de passe à cette cadence. **71,8 % des tentatives visent le compte** `root`, c'est-à-dire l'accès administrateur complet à la machine.

Aucune intrusion réussie n'apparaît dans les données. Le fichier contient 36 sessions ouvertes : je les ai vérifiées une par une, aucune ne se rattache aux adresses attaquantes. Le détail de cette vérification figure en partie 5, avec les réserves qui l'accompagnent.

Une seconde anomalie, sans lien avec l'attaque, est ressortie de la lecture manuelle : la rotation quotidienne des journaux échoue, **43 fois de suite**. Un serveur dont la journalisation

ne fonctionne plus perd la trace de ce qui lui arrive. C'est une défaillance de détection avant d'être une défaillance technique, et elle se traite en priorité.

Le risque principal n'est pas l'incident lui-même, c'est sa permanence : 47 sources réparties sur 42,7 jours décrivent un état, pas un événement. Les mesures prioritaires recommandées en partie 6 sont gratuites, tiennent en quelques lignes de configuration, et retirent l'essentiel de la surface exposée.

La démarche : trois temps, dans cet ordre

J'ai traité le même fichier trois fois, avec trois moyens différents : à la main, par script, puis dans un SIEM. L'ordre compte, et pas pour des raisons de confort — **on ne peut pas écrire une règle de détection avant d'avoir vu à quoi ressemble ce qu'on veut détecter.**

Temps	Moyen	Ce qu'il apporte	Sa limite
1	Lecture directe, puis Excel et Power Query	voir la donnée réelle, découvrir les motifs à chercher, juger la sévérité	ne passe pas l'échelle
2	Script Python	reproductible, instantané, exportable en CSV	ne trouve que ce qu'on lui a demandé de chercher
3	SIEM Splunk	agrégation, classement, visualisation sur la totalité	n'évalue pas la qualité de ce qu'on lui donne

Ce que chaque temps a trouvé seul

C'est le résultat méthodologique de ce travail, et il n'était pas prévu au départ. Chacun des trois moyens a produit un constat que les deux autres étaient **structurellement incapables** de produire.

Temps	Ce qu'il a trouvé, et que les autres ont manqué
Lecture directe	L'échec quotidien de `logrotate`. Cette ligne ne contient aucun mot-clé d'authentification : ni le script ni la requête SPL ne pouvaient la voir. Également le triage par sévérité, qui demande un jugement.
Script Python	L'absence totale de `Failed password` dans le fichier, et donc l'inadéquation des règles de détection usuelles à cette source. Également le repérage de la paire de lignes émises pour une même tentative, reconnaissables à leur numéro de processus identique.
SIEM Splunk	La vue d'ensemble : les 47 sources classées, et la réduction de 608 évènements à deux comportements.

Aucun des trois n'aurait suffi. L'automatisation ne remplace pas la lecture : elle la prolonge.

PARTIE 1 — Le temps manuel

1.1 Lire avant d'outiller

J'ai lu les 150 premières lignes du journal, une par une, dans un éditeur de texte. Aucun filtre, aucun comptage automatique. L'objectif n'était pas de produire un résultat mais de découvrir à quoi ressemble la donnée.

Ce que contiennent ces 150 lignes	Nombre
Échecs d'authentification	50
Mentions de compte inexistant (`user unknown`)	38
Adresses sources distinctes	6
Sessions ouvertes	11
Lignes FTP	32
Alertes `logrotate`	5

Constat 1 — Une rafale sur le compte `root`

Dix tentatives d'authentification en échec sur `root`, toutes depuis la même source, **toutes portant le même horodatage à la seconde : `Jun 15 02:04:59`.**

```
Jun 15 02:04:59 combo sshd(pam_unix)[20882]: authentication failure; ... rhost=220-135-151-1.hinet-ip.hinet.net  user=root
Jun 15 02:04:59 combo sshd(pam_unix)[20884]: authentication failure; ... rhost=220-135-151-1.hinet-ip.hinet.net  user=root
Jun 15 02:04:59 combo sshd(pam_unix)[20883]: authentication failure; ... rhost=220-135-151-1.hinet-ip.hinet.net  user=root
          [ ... sept lignes identiques, même seconde, mêmes paramètres ... ]
```

Le journal n'écrit pas une adresse numérique mais un nom de machine, `220-135-151-1.hinet-ip.hinet.net`. C'est la même machine que `220.135.151.1` : le fournisseur a composé le nom à partir de l'adresse en remplaçant les points par des tirets. Ne pas voir cette correspondance conduirait à compter deux fois la même source — et c'est exactement ce qui m'est arrivé plus loin, au paragraphe 1.3.

Conclusion : dix mots de passe dans la même seconde ne sont pas le fait d'un opérateur. **C'est un programme, et il visait l'accès administrateur.**

Constat 2 — Un balayage de noms de comptes

Trente-huit lignes portent la mention `user unknown` : le compte essayé n'existe pas sur la machine.

Ce n'est pas la même manœuvre que le constat 1. Attaquer `root`, c'est viser un compte dont on sait qu'il existe sur tout système Linux. Essayer des noms inexistants, c'est chercher **quels comptes existent**. Deux comportements distincts dans le même journal : de la reconnaissance d'un côté, de la force brute ciblée de l'autre.

Constat 3 — Une alerte système qui n'a rien à voir avec l'attaque

```
Jun 15 04:06:20 combo logrotate: ALERT exited abnormally with [1]
```

Cinq occurrences dans ces 150 lignes. **Quarante-trois dans le fichier entier**, une par jour aux alentours de 4 heures du matin, toujours le même message, sans aucun détail sur ce qui a échoué.

`logrotate` est le mécanisme qui archive et renouvelle les fichiers de journaux. S'il échoue tous les jours, leur conservation n'est plus garantie : risque de perte de traces, ou de saturation du disque. Or les journaux sont la matière première de toute investigation.

Constat 4 — Des échecs isolés, qui ne méritent pas d'alerte

D'autres échecs n'apparaissent **qu'une seule fois**, sans répétition. Vérification faite sur l'ensemble du fichier : **trois adresses sur 47 n'ont produit qu'une tentative**. Elles peuvent relever d'une faute de frappe d'un utilisateur légitime.

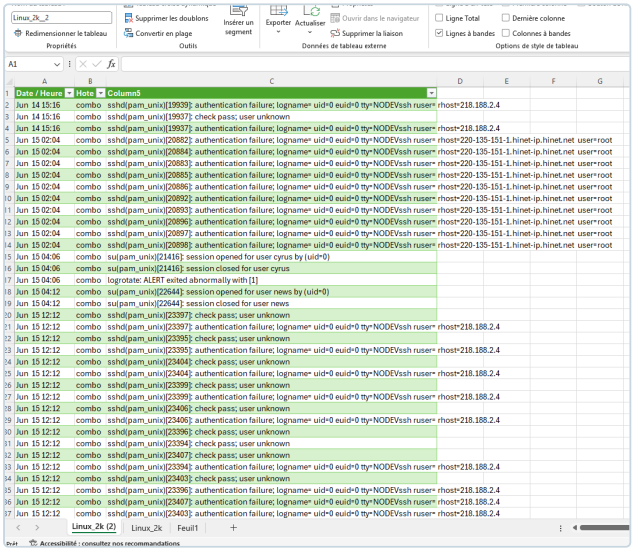
Les classer en sévérité faible évite de noyer les vraies alertes. C'est un échec isolé qui fait le bruit, et la répétition qui fait l'attaque.

1.2 Structurer dans Excel

La lecture donne des constats ; elle ne donne pas un document exploitable. J'ai donc construit un tableau d'analyse à six colonnes — date et heure, adresse source, compte visé, type d'évènement, nombre d'occurrences, sévérité — et j'y ai porté **quinze lignes de journal décomposées élément par élément**.

J'ai chronométré la saisie manuelle : cinq minutes pour dix lignes. Rapporté aux 2 000 lignes du fichier, cela représente **seize heures et quarante minutes** de travail. Ce chiffre n'est pas une anecdote : c'est la justification mesurée du passage à l'automatisation, et la réponse à la question « pourquoi scripter ? ».

Pour la suite j'ai utilisé **Power Query**, l'outil d'import et de transformation d'Excel, qui permet de nettoyer les données avant qu'elles n'arrivent dans la feuille. Le journal arrive éclaté en une dizaine de colonnes, l'horodatage réparti sur trois d'entre elles ; je les ai fusionnées en une seule colonne de date.



Résultat après transformation — l'horodatage reconstitué et les champs séparés proprement

1.3 Trois pièges d'outil, et ce qu'ils coûtent

Cette étape m'a appris plus que le résultat lui-même. **Les trois pièges rencontrés ont tous la même forme : l'outil produit une donnée plausible et fausse, sans rien signaler.**

Premier piège — Excel transforme une date en une autre date. Saisi tel quel, `Jun 14` devient « juin 2014 » : le jour du mois disparaît, remplacé par une année inventée. Mettre la colonne au format texte après coup ne restaure rien. La parade est de taper une apostrophe avant la saisie.

Deuxième piège — l'import détruit les secondes. Power Query a reconnu de lui-même la colonne des heures comme une donnée de type « heure » et l'a réécrite au format court : `15:16:01` **est devenu** `15:16`**.** Dans ce contexte précis, c'est une perte grave : le critère de détection d'une force brute se mesure en secondes — cinq échecs en moins de soixante secondes. Un horodatage réduit à la minute rend ce critère inapplicable. La parade est de supprimer l'étape de typage automatique avant de fusionner les colonnes.

Troisième piège — la même source comptée deux fois. Mon tableau portait l'adresse de la machine taïwanaise sous ses deux écritures, l'une avec des points, l'autre avec des tirets. Le tableau croisé dynamique ne compare pas des adresses : il compare des chaînes de caractères. Il en a donc fait deux sources distinctes, affichant huit occurrences d'un côté et une de l'autre **au lieu de neuf**.

Ce troisième piège est le plus instructif des trois, parce qu'il travaille contre la détection : une source répétée, découpée en deux entrées, passe sous le seuil d'alerte. **C'est la raison d'être de la normalisation des champs dans un SIEM** — unifier les écritures d'une même entité avant de compter quoi que ce soit.

1.4 La sévérité : un jugement, pas un calcul

Les deux dernières colonnes du tableau — le nombre d'occurrences et la sévérité — ne s'extraient pas du fichier. La première se compte, la seconde se décide.

La grille de référence prévoit quatre cas : compte `root` visé avec volume élevé en sévérité haute, comptes inexistants et alertes système en sévérité moyenne, échec isolé sur compte ordinaire en sévérité faible. **Elle ne tranche pas le cas que j'ai rencontré plusieurs fois : l'échec isolé sur** `root`**.**

J'ai donc posé ma règle et je l'ai appliquée uniformément : **le compte visé prime sur le volume**, le volume restant un facteur aggravant au-delà de cinq occurrences.

Le raisonnement : une tentative de connexion en `root` depuis Internet n'a aucune raison légitime d'exister. Sur un serveur correctement configuré, l'authentification directe de `root` par SSH est désactivée. Une seule tentative est donc déjà un signal, même sans répétition.

Cette règle vaut pour mes quinze lignes et elles seules. Elle est déduite de ce que j'ai observé dans cet échantillon, pas d'une analyse des 2 000 lignes. Quinze lignes ne permettent pas de hiérarchiser les sources à l'échelle du fichier : une adresse absente de l'échantillon peut être bien plus active que celles retenues. La partie 2 montre que c'est précisément ce qui s'est passé.

1.5 Qui est derrière l'adresse ?

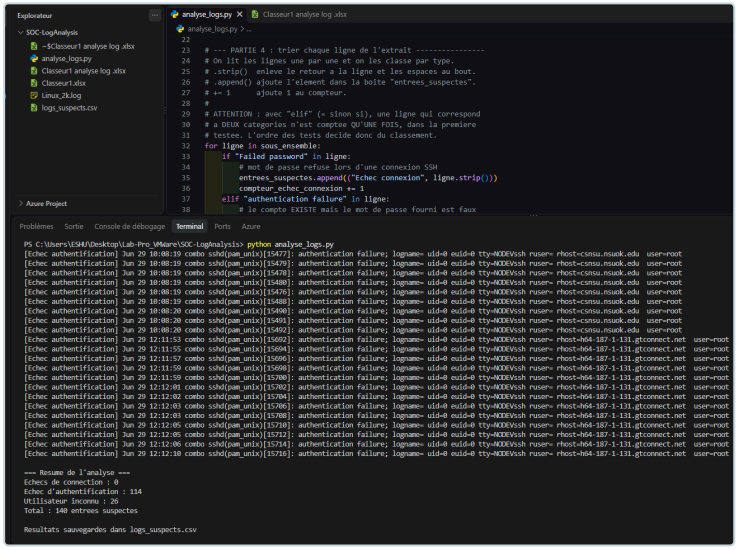
Le nom de machine `220-135-151-1.hinet-ip.hinet.net` porte une information que l'adresse numérique ne donne pas. Le domaine `hinet.net` appartient à **HiNet, le service Internet de Chunghwa Telecom, principal opérateur de Taïwan**. La structure du nom — l'adresse encodée avec des tirets — est celle que les opérateurs attribuent à leurs **lignes d'abonné grand public**, pas à des serveurs hébergés.

Ce qu'on peut en conclure, et ce qu'on ne peut pas : une machine d'abonné résidentiel qui lance dix tentatives sur le compte `root` d'un serveur à l'autre bout du monde n'est, selon toute vraisemblance, **pas pilotée par son propriétaire**. C'est le profil d'une machine compromise, enrôlée dans un réseau de machines zombies. En revanche rien ici ne permet d'attribuer l'attaque à qui que ce soit : l'origine géographique d'un paquet ne dit rien de l'origine de la décision.

PARTIE 2 — Le temps du script

Lire 150 lignes est possible. En lire 2 000 ne l'est pas. Le script transforme la lecture en règle appliquée mécaniquement : parcourir le fichier, classer chaque ligne selon les motifs repérés à l'œil, compter, exporter.

```
for ligne in sous_ensemble:
    if "Failed password" in ligne:
        entrees_suspectes.append(("Echec connexion", ligne.strip()))
    elif "authentication failure" in ligne:
        entrees_suspectes.append(("Echec authentification", ligne.strip()))
    elif "user unknown" in ligne or "invalid user" in ligne:
        entrees_suspectes.append(("Utilisateur inconnu", ligne.strip()))
```



Exécution du script — 140 entrées suspectes détectées et exportées en CSV

2.1 Première version : le résultat, et ce qu'il révèle

Catégorie	Nombre
Échecs de connexion (`Failed password`)	**0**
Échecs d'authentification	114
Utilisateurs inconnus	26
Total retenu sur 301 lignes	**140**

Trois constats, et aucun n'était attendu.

`Failed password` n'apparaît pas une seule fois dans tout le fichier. C'est pourtant le motif le plus cité dans la documentation sur la détection de force brute. L'explication est dans la source : ce

journal enregistre les évènements de `sshd(pam_unix)`, le module d'authentification du système, tandis que `Failed password` est le message du démon `sshd` lui-même. PAM dit la même chose avec d'autres mots.

La conséquence est opérationnelle et directe : **une règle de détection écrite autour de `Failed password` n'aurait rien vu de cette attaque.** Il ne suffit pas de connaître les bons mots-clés, il faut connaître le vocabulaire réel de la source qu'on surveille.

Cent quarante lignes ne font pas cent quarante évènements. Les 26 lignes `check pass; user unknown` portent toutes, sans exception, **le même numéro de processus** qu'une ligne `authentication failure` : ce sont les deux moitiés d'une même tentative. Le volume réel est d'environ 114 tentatives. Compter les lignes surestime l'attaque.

Cent soixante-une des 301 lignes n'entrent dans aucune catégorie, soit plus de la moitié. Le script les ignore en silence. Ce qu'une règle ne classe pas, elle ne le signale pas — et c'est exactement ainsi que l'alerte `logrotate` du constat 3 serait passée inaperçue.

2.2 Deuxième version : ce que la première ne voyait pas

Le script d'origine portait deux défauts. J'ai écrit une seconde version qui les corrige, et mesuré l'écart plutôt que de l'effacer.

	Défaut	Correction
1	Le périmètre `logs[199:500]` ne lit que **301 lignes sur 2 000**, soit 15 % du fichier	lire le fichier entier
2	La comparaison `"user unknown" in ligne` distingue les majuscules	comparer en minuscules

Résultat des deux versions sur le même fichier :

	Version 1	Version 2
Périmètre lu	301 lignes (15 %)	**2 000 lignes (100 %)**
Échecs d'authentification	114	**490**
Utilisateurs inconnus	26	**118**
Total retenu	140	**608**
Adresses sources identifiées	12	**47**

Le constat décisif est ailleurs que dans ces totaux. L'adresse la plus active du fichier, `150.183.249.110` avec ses 80 tentatives, **est absente du périmètre de la version 1**. La tête du classement de la version 1 est une autre adresse, à 22 tentatives. Autrement dit : le script d'origine fonctionnait parfaitement, et **ne voyait pas l'attaquant principal.**

C'est aussi la démonstration en acte de la réserve que j'avais posée au paragraphe 1.4 : l'adresse taïwanaise relevée à la main n'apparaît pas non plus dans le périmètre de la version 1. Un échantillon ne hiérarchise rien.

L'effet de la casse, mesuré à périmètre égal, est d'une seule ligne : 607 contre 608. Cette ligne unique vaut d'être citée, car elle sert deux fois :

```
Jun 18 02:23:10 combo ftpd[31277]: User unknown timed out after 900 seconds at Sat Jun 18 02:23:10 2005
```

Elle est émise par `ftpd` et non par SSH, elle écrit `User` avec une majuscule — et **c'est la seule ligne du fichier entier qui porte une année réelle : 2005.** Elle reviendra en partie 7, à propos de la datation.

Un écart d'une ligne est un détail dans cet exercice. En production, c'est un **faux négatif silencieux** : une variation d'écriture suffirait à passer sous le radar.

2.3 Ce que le script a coûté, ce qu'il a rapporté

	À la main	Par script
Les 301 lignes de l'extrait	2 h 30, au rythme mesuré de 5 min pour 10 lignes	**moins d'une seconde**
Les 2 000 lignes du fichier	16 h 40	**moins d'une seconde**

Et une vérification qui compte plus que le gain de temps : **les 608 lignes retenues par la version 2 correspondent exactement aux 608 évènements que trouvera Splunk en partie 3.** Deux outils indépendants, deux langages différents, le même chiffre. C'est ce qui permet de faire confiance aux deux.

PARTIE 3 — Le temps du SIEM

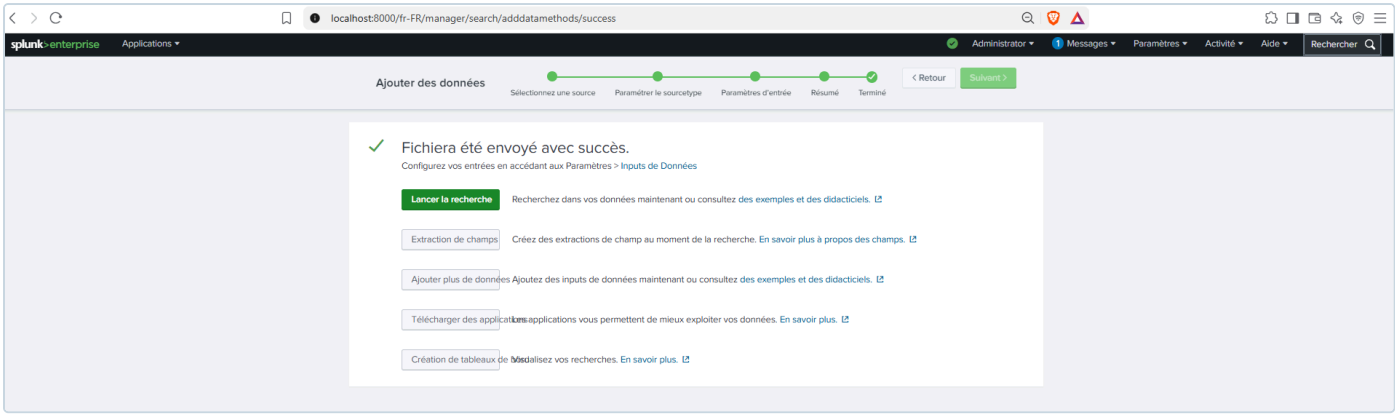
3.1 Ingestion, et deux décisions à prendre

Le fichier a été chargé dans Splunk Enterprise 10.6, index `main`, avec un type de source dédié nommé `Linux`. Deux points ont demandé une décision.

Le nom du fichier. Splunk échoue à l'envoi si le nom contient certains caractères ; le fichier a donc été renommé `Linux2k.log` pour cette étape seulement. Il garde son nom d'origine `Linux_2k.log` partout ailleurs. C'est la raison pour laquelle les deux écritures apparaissent dans ce rapport : la première dans les requêtes SPL, la seconde partout ailleurs.

Le champ `host`. La procédure prévoit d'y porter le nom du poste qui importe les données. Je l'ai renseigné à `combo`, nom du serveur qui a réellement produit ces journaux, lisible dans chaque ligne du fichier. Le champ `host` doit désigner la machine émettrice : une provenance fausse rend la donnée inexploitable pour l'analyste qui cherchera plus tard « les attaques sur `combo` ».

État vérifié après ingestion : 2 000 évènements, `host=combo`, `sourcetype=Linux`.



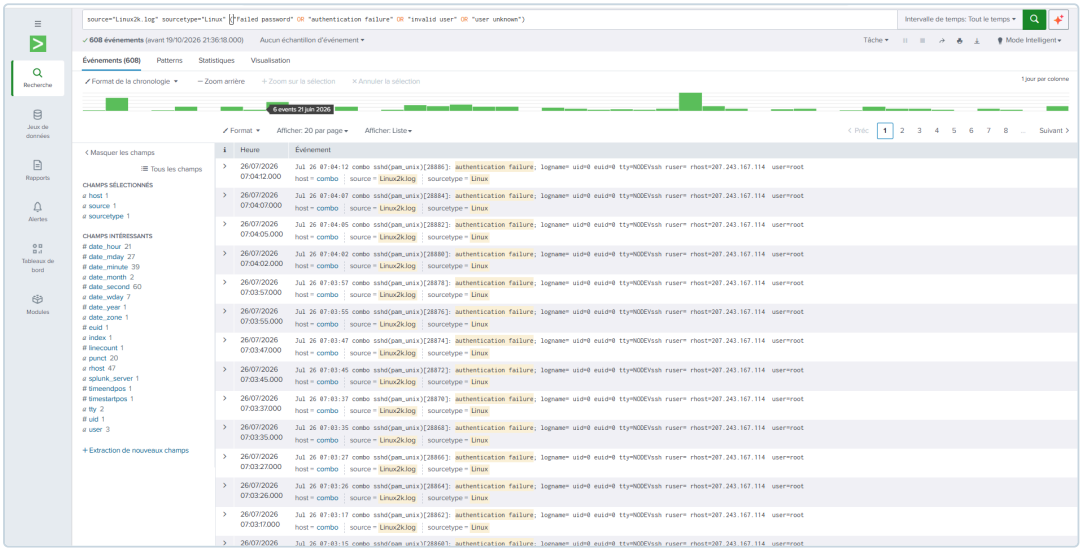
Confirmation de l'ingestion dans Splunk

3.2 Les recherches

Première recherche, sur les quatre motifs d'échec, période réglée sur « Tout le temps » :

```
source="Linux2k.log" sourcetype="Linux" ("Failed password" OR "authentication failure" OR "invalid user" OR "user unknown")
```

608 évènements — le chiffre de la version 2 du script.



Vue Événements — les 608 évènements correspondant à la recherche

Seconde recherche, pour établir le classement des sources :

```
source="Linux2k.log" sourcetype="Linux" "authentication failure" | stats count by rhost | sort -count
```

47 adresses, avec `150.183.249.110` en tête à **80 tentatives**.

Cette requête filtre sur `authentication failure` seul, alors que la précédente interrogeait quatre motifs. Ce n'est pas une incohérence : la question a changé. On cherche désormais **qui**, et seul ce motif porte le champ `rhost`, l'adresse de la machine distante. Les lignes `check pass; user unknown` ne nomment personne. On ne filtre pas sur ce qui est le plus fréquent, mais sur ce qui porte la réponse à la question posée.

Le signe `-` de `sort -count` n'est pas un détail de confort : sans lui le classement s'inverse, l'attaquant principal se retrouve en dernière page, et s'affichent en tête les adresses n'ayant tenté qu'une seule fois.

3.3 Quatre vues, quatre questions

Les quatre onglets de Splunk ne relancent aucune recherche. Ce sont quatre angles sur les mêmes 608 évènements, et c'est la question posée qui détermine l'onglet à ouvrir.

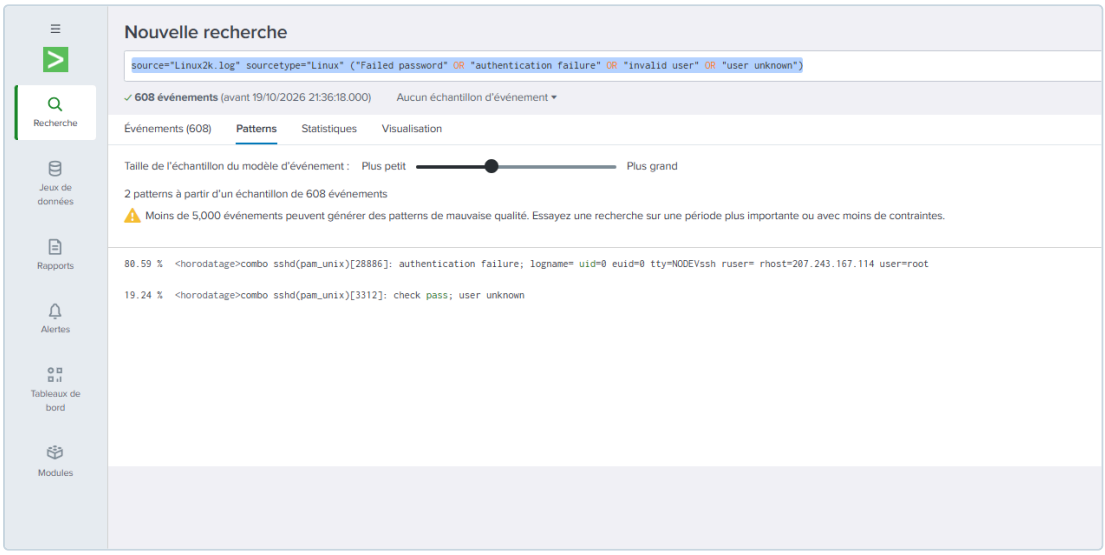
Onglet	La question à laquelle il répond
Événements	que dit exactement cette ligne ?
Patterns	qu'est-ce qui se passe, globalement ?
Statistiques	qui, et combien de fois ?
Visualisation	comment le montrer à quelqu'un ?

Patterns et Statistiques sont souvent confondus : **Patterns regroupe sur la forme de la phrase, Statistiques compte sur la valeur d'un champ.**

La vue Patterns

Splunk remplace par un trou ce qui change d'une ligne à l'autre, puis rassemble les lignes dont le reste est identique.

Part	Motif
80,59 %	`combo sshd(pam_unix)[…]: authentication failure; … rhost=… user=root`
19,24 %	`combo sshd(pam_unix)[…]: check pass; user unknown`



Vue Patterns — deux motifs couvrent 99,83 % des évènements

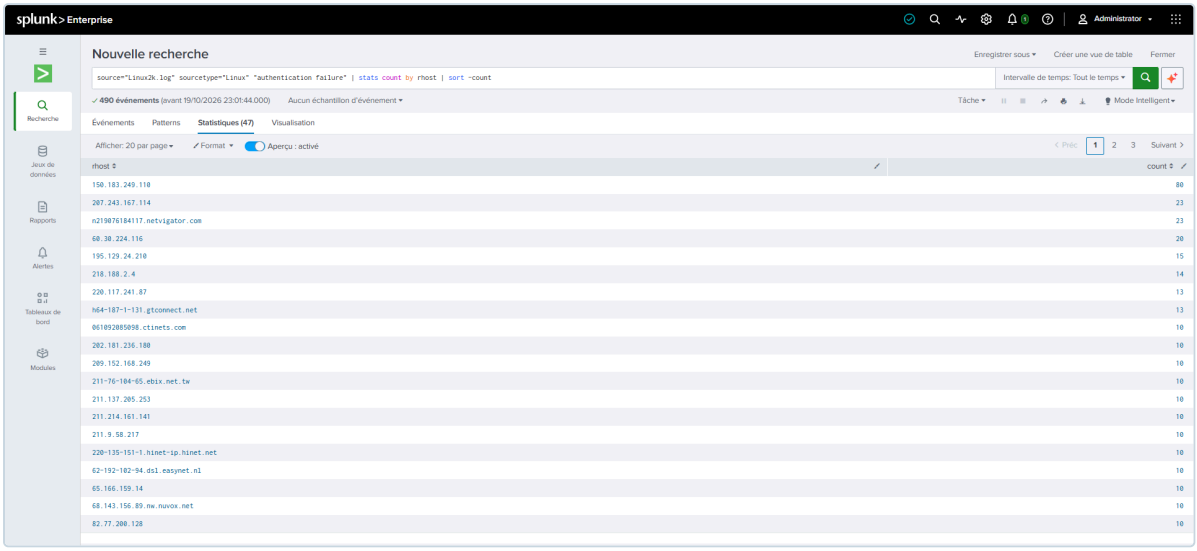
Deux motifs couvrent 99,83 % des évènements. Une activité humaine produit des erreurs variées ; cette uniformité est la signature d'un programme. La monotonie du résultat n'est pas un défaut de présentation — elle est la preuve.

Ces deux motifs sont exactement la paire que j'avais identifiée à la main dans la partie 2, en rapprochant les numéros de processus. Deux méthodes indépendantes, le même résultat.

Un piège de lecture mérite d'être signalé. La ligne de motif affiche une adresse en exemple, `rhost=207.243.167.114`. **C'est un représentant du groupe, pas une caractéristique de ses 490 membres** : cette adresse n'apparaît que 23 fois sur l'ensemble, et elle n'est pas la

plus active. Prendre cet exemple pour le résultat conduirait à désigner la mauvaise source dans le rapport. Le classement réel ne s'obtient que par l'onglet Statistiques.

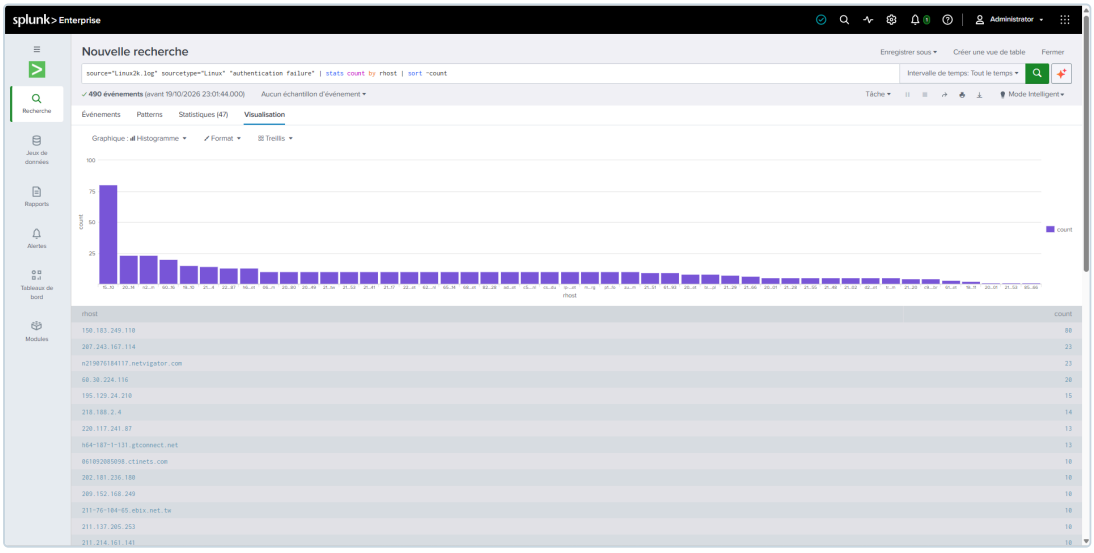
La vue Statistiques



Vue Statistiques — le classement des 47 adresses sources par nombre de tentatives

Un enseignement d'interface, qui m'a coûté plusieurs vérifications : **un clic sur un en-tête de colonne écrase silencieusement le `| sort -count` de la requête.** Les chiffres affichés restent justes, mais l'ordre devient celui du clic. J'ai d'abord capturé un tableau trié à l'envers, qui montrait en tête les adresses n'ayant tenté qu'une seule fois et où l'attaquant principal n'apparaissait pas. La donnée était exacte et la démonstration fausse. La parade fiable est de relancer la recherche plutôt que de viser l'en-tête.

La vue Visualisation



Histogramme des tentatives par adresse source — une barre domine toutes les autres

L'histogramme rend la conclusion lisible en une seconde : une barre écrase toutes les autres. Ce qui demandait un tableau de 47 lignes tient dans une image qu'un responsable non technique comprend sans explication. Cette vue sert à communiquer un résultat déjà établi, pas à le chercher.

J'ai écarté le graphique en courbes, proposé par l'interface. Une courbe relie les points : elle affirme une continuité entre deux valeurs voisines. Or l'axe horizontal porte des adresses IP, qui n'ont ni ordre naturel ni distance entre elles — la pente n'y signifie rien. L'histogramme est la forme juste pour comparer des catégories indépendantes ; la courbe ne conviendrait que si l'axe portait le temps.

PARTIE 4 — Findings

F1 — Force brute SSH automatisée, mesurée à deux échelles

Indicateur	Valeur mesurée
Échecs d'authentification SSH	**490**
Évènements correspondant aux motifs d'échec	**608**
Adresses sources distinctes	**47**
Tentatives de l'adresse la plus active (`150.183.249.110`)	**80**
Durée de cette rafale	**95 secondes**, le 10 juillet de 16:01:43 à 16:03:18
Pic par minute	**50 tentatives**
Pic par seconde (`220.135.151.1`)	**10 tentatives dans la même seconde**

La rafale de `150.183.249.110` est continue : aucune pause supérieure à une minute sur ses 95 secondes d'activité.

F2 — Le compte `root` est la cible principale, à 71,8 %

Compte visé	Tentatives	Part
`root`	**351**	**71,8 %**
compte inexistant, aucun nom retenu	117	23,9 %
`guest`	17	3,5 %
`test`	4	0,8 %

`root` détient tous les droits sur un système Linux : le compromettre, c'est compromettre la machine entière. Près des trois quarts de l'effort adverse portent sur ce seul compte.

Les 117 tentatives sans nom de compte correspondent au balayage de noms d'utilisateurs, comportement d'un outil générique lancé sans reconnaissance préalable.

F3 — Une attaque distribuée, et permanente

Les dix adresses les plus actives, sur 47 :

Adresse source	Tentatives
`150.183.249.110`	**80**
`n219076184117.netvigator.com`	23
`207.243.167.114`	23
`60.30.224.116`	20
`195.129.24.210`	15
`218.188.2.4`	14
`h64-187-1-131.gtconnect.net`	13
`220.117.241.87`	13
`220-135-151-1.hinet-ip.hinet.net`	10
`061092085098.ctinets.com`	10

Deux lectures se superposent, et il faut les distinguer :

- à l'échelle d'une adresse, l'attaque est brève et violente — 80 tentatives en 95 secondes ;
- **à l'échelle du serveur, elle ne s'arrête jamais** — 47 sources réparties sur six semaines.

Ce n'est donc pas un incident ponctuel, c'est un état permanent. Un rapport qui ne retiendrait que la rafale du 10 juillet manquerait l'essentiel du risque.

F4 — Aucun `Failed password`, aucun `invalid user`, aucun `Accepted`

Motif recherché	Occurrences sur 2 000 lignes
`authentication failure`	490
`user unknown`	118
`Failed password`	**0**
`invalid user`	**0**
`Accepted` — connexion réussie	**0**

Voir le premier constat de la partie 2 : une règle bâtie sur `Failed password` n'aurait rien détecté.

F5 — SSH n'est pas le seul service exposé

Service	Constat mesuré
`ftpd`	**909 connexions entrantes depuis 38 sources distinctes**, la plus active totalisant 46 connexions. C'est le service le plus volumineux du fichier, 916 lignes, et il n'a pas été analysé.
`klogind`	**23 échecs d'authentification Kerberos**, tous depuis une source unique : `163.27.187.39`

Ces lignes ne prouvent pas une attaque FTP : ce sont des connexions, pas des échecs. Mais elles établissent un fait — **la surface exposée dépasse SSH, et 45 % du fichier n'a pas été regardé.**

F6 — La rotation des journaux échoue quotidiennement

Quarante-trois occurrences de `logrotate: ALERT exited abnormally with [1]`, une par jour vers 4 heures du matin, sur toute la période, toujours le même message et sans aucun détail.

Sans lien avec l'attaque, mais à traiter en priorité : si la rotation des journaux échoue, leur conservation n'est pas garantie. **Une défaillance de journalisation est une défaillance de détection.**

Ce finding n'a pas été produit par un outil. La ligne `logrotate` ne contient aucun mot-clé d'authentification : ni le script ni la requête SPL ne pouvaient la voir. Elle vient de la lecture directe.

PARTIE 5 — L'attaque a-t-elle abouti ?

C'est la question que doit trancher un analyste, et elle ne se déduit pas du nombre de tentatives. 490 échecs prouvent une tentative. Ils ne disent rien d'une réussite.

Vérification	Résultat
Lignes `Accepted password` ou `Accepted publickey`	**0**
Connexions réussies citant une des 47 adresses attaquantes	**0**
Sessions SSH ouvertes dans le fichier	**36**, toutes pour un compte nommé `test`

Ces 36 sessions ont exigé une vérification, puisque `test` figure parmi les comptes attaqués. **Elles ne sont pas le fait de l'attaquant**, et trois éléments concordants l'établissent :

Élément	Constat
Chronologie	Les sessions `test` commencent le **17 juin**. Le compte `test` n'a été attaqué que le **8 juillet**, depuis `212.0.132.20`. Trois semaines séparent les deux : l'effet précéderait la cause.
Comportement	Les sessions s'ouvrent et se ferment **dans la même seconde**, par rafales de dix. Un intrus qui obtient un accès l'exploite et reste ; ce profil est celui d'un script local automatisé.
Origine	Aucune de ces lignes ne porte d'adresse source : rien ne les rattache à une connexion distante.

Conclusion, et sa réserve

> Rien dans les journaux analysés ne montre d'intrusion réussie. Les tentatives ont échoué.

Cette conclusion porte sur ce que la preuve montre, et pas au-delà. Deux réserves l'accompagnent : le fichier est un **échantillon de 2 000 lignes**, non la totalité des journaux du serveur ; et les lignes de session **ne portent pas d'adresse source**, ce qui limite ce qu'on peut en établir.

L'absence de preuve d'intrusion n'est pas une preuve d'absence d'intrusion. Un rapport qui affirmerait « le serveur n'a pas été compromis » dépasserait ce que les données autorisent.

Classification

Campagne de force brute SSH automatisée, distribuée et continue, ciblant prioritairement le compte `root` du serveur `combo`. Aucune compromission établie sur la

période analysée.

MITRE ATT&CK : T1110 — Brute Force, sous-technique **T1110.001 — Password Guessing** : l'adversaire essaie de nombreux mots de passe contre un compte qu'il sait exister, `root`, plutôt que peu de mots de passe contre de nombreux comptes.

Stade dans la chaîne d'attaque : accès initial, tentative non aboutie. Il n'y a eu ni exécution, ni persistance, ni élévation de privilège. Rien ne montre que l'adversaire soit allé au-delà de la porte d'entrée.

PARTIE 6 — Recommandations

Classées par rapport entre l'effet obtenu et l'effort demandé.

R1 — Interdire la connexion SSH directe au compte `root` *(priorité haute)*

```
# /etc/ssh/sshd_config
PermitRootLogin no
```

351 des 490 tentatives visent `root`. Cette seule ligne annule 71,8 % de la surface d'attaque observée. L'administration passe alors par un compte nommé suivi de `sudo`, ce qui rend en outre chaque action traçable à une personne.

R2 — Déployer `fail2ban` *(priorité haute)*

Blocage automatique et temporaire d'une adresse après un nombre d'échecs donné.

39 des 47 adresses dépassent 5 échecs en 60 secondes. Avec ce seuil, **83 % des sources attaquantes auraient été bloquées sans aucune intervention humaine.**

R3 — Créer une alerte SIEM : 5 échecs en 60 secondes depuis une même adresse

```
index=main sourcetype="Linux" "authentication failure"
| bin _time span=60s
| stats count by _time, rhost
| where count >= 5
```

Le seuil n'est pas repris d'un guide, il est **calibré sur les données réelles** : 39 sources sur 47 le franchissent, et le pic observé atteint 50 échecs par minute. Il détecte donc largement sans produire de bruit.

R4 — Filtrer au pare-feu, sans en faire la mesure principale

Le blocage des adresses les plus actives apporte un gain immédiat. Mais **avec 47 sources distinctes renouvelées sur six semaines, le blocage manuel adresse par adresse est une course perdue** : il traite le symptôme du jour. Le blocage dynamique (R2) et la réduction de surface (R1) sont structurellement plus efficaces. Privilégier, lorsque c'est possible, une restriction par liste d'autorisation sur le port 22.

R5 — Supprimer ou renommer le compte `test`

Un compte nommé `test` joignable en SSH figure en tête des dictionnaires d'attaque, et il a effectivement été visé le 8 juillet. S'il n'a pas d'usage établi, le supprimer ; sinon, le renommer et lui interdire l'accès SSH.

R6 — Investiguer l'échec quotidien de `logrotate`

```
logrotate -d /etc/logrotate.conf
```

43 échecs consécutifs, un par jour. Tant que la rotation ne fonctionne pas, la conservation des journaux n'est pas assurée — et sans journaux, aucune investigation n'est possible.

R7 — Étendre la surveillance au-delà de SSH

`ftpd` est le service le plus volumineux du fichier, 916 lignes, et n'est pas surveillé, tandis que `klogind` enregistre 23 échecs Kerberos depuis une source unique. La supervision doit couvrir tous les services exposés, pas seulement celui par lequel on a commencé à chercher.

R8 — Imposer des mots de passe robustes et supprimer les identifiants d'usine

Les 351 tentatives sur `root` ne réussissent que si le mot de passe est devinable. Un compte qui existe avec un mot de passe par défaut tombe en quelques secondes sous une attaque de ce type.

PARTIE 7 — Limites de l'analyse

Limite	Portée
Échantillon	2 000 lignes, non la totalité des journaux du serveur.
Périmètre fonctionnel	Seul SSH a été analysé en profondeur. `ftpd` représente 916 lignes, **45 % du fichier**, et reste inexploré.
Horodatage non fiable	Le format syslog **ne porte pas l'année**. Splunk a comblé ce vide avec l'année courante et date les évènements de **2026**. Une seule ligne du fichier porte une année réelle — celle citée en partie 2 : **2005**. **Vingt et un ans d'écart** entre la date affichée par le SIEM et la date réelle. Toute conclusion fondée sur l'année serait fausse par construction.
Sensibilité à la casse	Le script distingue les majuscules, Splunk non. Écart mesuré : **607 contre 608**. Détail ici, faux négatif silencieux en production.
Enrichissement absent	Aucune corrélation avec une base de réputation d'adresses. L'analyse du nom de domaine en partie 1.5 reste une déduction, pas un constat vérifié auprès d'une source externe.
Secondes perdues côté Excel	Le typage automatique à l'import a réduit les horodatages à la minute dans le tableau Excel. Les mesures à la seconde de ce rapport proviennent du fichier d'origine, pas de ce tableau.

Deux incidents d'outil rencontrés en cours de route méritent d'être consignés, parce qu'ils ont failli fausser le résultat. **Un aperçu d'import n'est pas une ingestion** : le premier envoi vers Splunk n'avait rien stocké alors que l'écran affichait les données, et seule une recherche dans l'index l'a établi. **Une commande de suppression peut échouer en silence** : une première

tentative a laissé 4 000 évènements en double dans l'index, détectés uniquement parce qu'un contrôle a été fait après coup.

Suites de l'investigation

Ce que je ferais ensuite, dans l'ordre :

1. **Obtenir les journaux complets du serveur**, et non un échantillon, pour confirmer l'absence d'intrusion sur toute la période.
2. **Analyser les 916 lignes** `ftpd`, seul service plus volumineux que SSH dans ce fichier et jamais examiné.
3. **Interroger une base de réputation** sur les 47 adresses, pour distinguer les machines compromises des infrastructures dédiées à l'attaque.
4. **Vérifier les 23 échecs Kerberos de** `klogind` depuis `163.27.187.39`, qui forment un comportement isolé et cohérent.
5. **Mettre l'alerte R3 en production** et mesurer son taux de faux positifs sur une semaine réelle avant de la généraliser.

Rapport produit dans le cadre du projet 2 — Starter Cyber by Jodry. Jeu de données public LogHub (`https://github.com/logpai/loghub`) : aucun système réel n'a été attaqué, aucune donnée personnelle n'est traitée.